# Malabar Polytechnic Campus

# Lab Manual

***for***

# Python Programming Lab (3347)

Artificial Intelligence and Machine Learning

(Semester 3)

**Prepared by Department of AIM**

## 1 . Area Of Triangle

**Aim** : Write a program to Find the area of a right angled triangle

**Program**

```
b=int(input("Enter the base"))
h=int(input("Enter the height"))
c=b*h
area=c/2
print("Area of right angle triangle is:",area,"sqcm")
```

**Sample output**

Enter the base : 5

Enter the height : 10

Area of right angled triangle is: 25 sqcm

## 2 . Largest Among Three Numbers

**Aim** : Write a program to Find the greatest among three numbers

**Program**

```
a=int(input("Enter first number"))
b=int(input("Enter second number"))
if a>b and a>c:
  print(“largest number is “,a)
elif b>c:
  print(“the largest number is “,b)
else:
  print(“the largest number is “,c)
```

**Sample output**

```
Enter first number :10
Enter second number : 5
Enter third number : 2
largest number is : 10
```

## 3 . Palindrome Numbers

**Aim** : Write a program to check whether a given number is palindrome or not

**Program**

```
n = int(input("enter a number"))
rev = 0
temp = n
while temp > 0:
   rem = temp % 10
   rev = rev * 10 + rem
   temp = temp // 10
if temp == num :
   print("palindrome")
else:
  print("not palindrome")
```

**Sample output**

```
Enter the number : 121
palindrome
```

## 4 . Fibonacci Series

**Aim** : Write a program to print Fibonacci series

**Program**

```
n=int(input("enter your number of  terms"))
a=0
b=1
print(a)
print(b)
for i in range(2,n):
    c=a+b
    print(c)
    a=b
    b=c
```

**Sample output**

Enter your number of terms :7

0 1 1 2 3 5 8

## 5 . Vowels And Consonants

**Aim** : Write a program to count number of vowels and consonants in a string

**Program**

```
s = str(input("enter a s tring"))
l = len(s)
c=0
v=0
for i in range (l):
  if s[i] == 'a' or s[i] == 'e' or s[i] == 'i' or s[i] == 'o' or s[i] == 'u' or s[i] == 'A' or s[i] == 'E' or s[i] == 'I' or s[i] == 'O' or s[i] == 'U' :
    v = v+ 1
elif s[i].isalpha():
   c = c + 1
print("Number of vowels is ",v)
print("Number of consonants is ",c)
```

**Sample output**

```
Enter a string : Hello
Number of vowels is 2
Number of consonants is 3
```

## 6 . Sorting

Aim : Write a program to Sort the elements of a list using function

**Program**

```
def bubble_sort(arr):
    n = len(arr)

    for i in range(n):
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]

    return arr

sample_list = [64, 34, 25, 12, 22, 11, 90]
sorted_list = bubble_sort(sample_list)
print(f"Sorted list in non-decreasing order: {sorted_list}")
```

**Sample output**

```
Sorted list in non-decreasing order: [11, 12, 22, 25, 34, 64, 90]
```

## 7 . Maximum And Minimum Element Of  A Tuple

Aim : Write a program to find Maximum and minimum elements of a tuple using function

**Program**

```
def find_max_min(tup):
    if not tup:
        return None, None
    max_element = tup[0]
    min_element = tup[0]

    for num in tup:
        if num > max_element:
            max_element = num
        if num < min_element:
            min_element = num
    return max_element, min_element

sample_tuple = (3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5)
max_element, min_element = find_max_min(sample_tuple)
print(f"The maximum element in the tuple is: {max_element}")
print(f"The minimum element in the tuple is: {min_element}")
```

**Sample output**

```
The maximum element in the tuple is: 9
The minimum element in the tuple is: 1
```

## 8 . List Of Frequencies

Aim : Write a program to create list of frequencies in non-increasing order using function

**Program**

```
def frequency_list(arr):
    if not arr:
        return []
    frequency_dict = {}

    for item in arr:
        if item in frequency_dict:
            frequency_dict[item] += 1
        else:
            frequency_dict[item] = 1
    frequencies = list(frequency_dict.values())

    for i in range(len(frequencies)):
        for j in range(i + 1, len(frequencies)):
            if frequencies[i] < frequencies[j]:
                # Swap the elements
                frequencies[i], frequencies[j] = frequencies[j], frequencies[i]

    return frequencies
sample_list = [3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5]
frequencies = frequency_list(sample_list)
print(f"List of frequencies in non-increasing order: {frequencies}")
```

**Sample output**

```
List of frequencies in non-increasing order: [3, 2, 2, 1, 1, 1, 1]
```

## 9 . Frequency Of Characters Of A File

Aim : Write a program to find Frequency of characters in a given file

**Program**

```
def character_frequency(filename):
    try:
        with open(filename, 'r') as file:
            text = file.read()
        frequency_dict = {}
        for char in text:
            if char in frequency_dict:
                frequency_dict[char] += 1
            else:
                frequency_dict[char] = 1


        return frequency_dict
    except FileNotFoundError:
        print(f"The file {filename} does not exist.")
        return {}
filename = 'file.txt'  # Replace with your file name
f = open(filename , 'x')
f.write("hello world")
f.close()
frequencies = character_frequency(filename)
print(f"Frequencies of characters in the file: {frequencies}")
```

**Sample output**

```
Frequencies of characters in the file: {'h': 1, 'e': 1, 'l': 3, 'o': 2, ' ': 1, 'w': 1, 'r': 1, 'd': 1}
```

## 10 . Exception Handling

Aim : Write a program to handle exception in the case of voters age validity If age < 0 raise an exception

**Program**

```
def check_voter_age(age):
    if age < 0:
        raise ValueError("Age cannot be negative.")
    elif age < 18:
        print("You are not eligible to vote.")
    else:
        print("You are eligible to vote.")
try:
    age = int(input("Enter your age: "))
    check_voter_age(age)
except ValueError as e:
    print(f"Error: {e}")
except TypeError:
    print("Please enter a valid integer for age.")
```

**Sample output**

Enter your age: -5

Error: Age cannot be negative.

## 11 . Student Class

Aim : Write a program to create a student class and create following methods

setMarks() – to update marks

getMarks() – to get individual marks

avgMarks() – to find average of marks

showResults() – to show results of each student

**Program**

```
class Student:
    def __init__(self, name):
        self.name = name
        self.marks = {}

    def setMarks(self, subject, mark):
        self.marks[subject] = mark

    def getMarks(self, subject):
        return self.marks.get(subject, "No marks available for this subject.")

    def avgMarks(self):
        if not self.marks:
            return "No marks available."
        total_marks = sum(self.marks.values())
        average_marks = total_marks / len(self.marks)
        return average_marks

    def showResults(self):
        if not self.marks:
            print(f"No marks available for {self.name}.")
```

```
            return

        print(f"Results for {self.name}:")
        for subject, mark in self.marks.items():
            print(f"{subject}: {mark}")

        average_marks = self.avgMarks()
        print(f"Average Marks: {average_marks}")

student1 = Student("John Doe")
student1.setMarks("Math", 85)
student1.setMarks("Science", 90)
student1.setMarks("English", 88)

print(student1.getMarks("Math"))  # Output: 85
print(student1.getMarks("History"))  # Output: No marks available for this subject.

print(student1.avgMarks())  # Output: 87.66666666666667

student1.showResults()
```

**Sample output**

```
85
No marks available for this subject.
87.66666666666667
Results for John Doe:
Math: 85
Science: 90
English: 88
Average Marks: 87.66666666666667
```

## 12 . Inheritance

Aim : Write a program to create a **vehicle** class with variables name , speed , mileage and methods setCapacity() and getFare() , create a class **bus** that inherits from the Vehicle class.

**Program**

```
class Vehicle:
    def __init__(self, name, speed, mileage):
        self.name = name
        self.speed = speed
        self.mileage = mileage
        self.capacity = 0

    def setCapacity(self, capacity):
        self.capacity = capacity
    def getFare(self):
        return self.capacity * 100  # Assuming fare is 100 per passenger

class Bus(Vehicle):
    def getFare(self):
        base_fare = super().getFare()
        return base_fare * 1.1  # Adding 10% maintenance charge

vehicle1 = Vehicle("Car", 150, 15)
vehicle1.setCapacity(5)
print(f"Vehicle Name: {vehicle1.name}")
print(f"Speed: {vehicle1.speed} km/h")
print(f"Mileage: {vehicle1.mileage} km/l")
print(f"Capacity: {vehicle1.capacity}")
print(f"Fare: {vehicle1.getFare()}")

bus1 = Bus("School Bus", 80, 10)
```

```
bus1.setCapacity(50)
print(f"\nBus Name: {bus1.name}")
print(f"Speed: {bus1.speed} km/h")
print(f"Mileage: {bus1.mileage} km/l")
print(f"Capacity: {bus1.capacity}")
print(f"Fare: {bus1.getFare()}")
```

**Sample output**

```
Vehicle Name: Car
Speed: 150 km/h
Mileage: 15 km/l
Capacity: 5
Fare: 500
Bus Name: School Bus
Speed: 80 km/h
Mileage: 10 km/l
Capacity: 50 Fare: 5500.0
```

## 13 . Numpy

Aim : Write a program to create a matrix using numpy and do the following operations - Perform matrix addition , subtraction and multiplication

- Find transpose of matrix –

- Calculates the inverse of matrix

- Calculates the determinant of matrix

- Transforms a matrix in to 1d array (flatten)

**Program**

```
import numpy as np


# Create matrices
matrix1 = np.array([[1, 2], [3, 4]])
matrix2 = np.array([[5, 6], [7, 8]])


# Perform matrix addition
addition = np.add(matrix1, matrix2)


# Perform matrix subtraction
subtraction = np.subtract(matrix1, matrix2)


# Perform matrix multiplication
multiplication = np.dot(matrix1, matrix2)


# Find transpose of matrix1
transpose = np.transpose(matrix1)


# Calculate the inverse of matrix1
inverse = np.linalg.inv(matrix1)


# Calculate the determinant of matrix1
determinant = np.linalg.det(matrix1)
```

```
# Transform matrix1 into a 1D array (flatten)
flattened = matrix1.flatten()

# Display results
print("Matrix 1:")
print(matrix1)
print("\nMatrix 2:")
print(matrix2)
print("\nMatrix Addition:")
print(addition)
print("\nMatrix Subtraction:")
print(subtraction)
print("\nMatrix Multiplication:")
print(multiplication)
print("\nTranspose of Matrix 1:")
print(transpose)
print("\nInverse of Matrix 1:")
print(inverse)
print("\nDeterminant of Matrix 1:")
print(determinant)
print("\nFlattened Matrix 1:")
print(flattened)
```

**Sample output**

```
Matrix 1:
[[1 2]
 [3 4]]

Matrix 2:
```

[[5 6]
 [7 8]]

Matrix Addition:
[[ 6  8]
 [10 12]]

Matrix Subtraction:
[[-4 -4]
 [-4 -4]]

Matrix Multiplication:
[[19 22]
 [43 50]]

Transpose of Matrix 1:
[[1 3]
 [2 4]]

Inverse of Matrix 1:
[[-2.   1. ]
 [ 1.5 -0.5]]

Determinant of Matrix 1:
-2.0000000000000004

Flattened Matrix 1:
[1 2 3 4]

## 14 . Panda Module

Aim : Write a python program to read a .csv file using panda module and print the first and last five lines of a file

**Program**

```
import pandas as pd


# Read the CSV file
filename = 'example.csv'  # Replace with your CSV file name
df = pd.read_csv(filename)


# Print the first five lines of the file
print("First five lines of the file:")
print(df.head())


# Print the last five lines of the file
print("\nLast five lines of the file:")
print(df.tail())
```

sample.csv

```
Name,Age,City
Alice,30,New York
Bob,25,Los Angeles
Charlie,35,Chicago
David,40,Houston
Eve,28,Phoenix
Frank,33,Philadelphia
Grace,45,San Antonio
Hannah,27,San Diego
Ivan,50,Dallas
Judy,29,San Jose
```

**Sample Output**

First five lines of the file:

```
      Name  Age         City
0    Alice   30     New York
1      Bob   25  Los Angeles
2  Charlie   35      Chicago
3    David   40      Houston
4      Eve   28      Phoenix
```

Last five lines of the file:

```
     Name  Age         City
5   Frank   33  Philadelphia
6   Grace   45   San Antonio
7  Hannah   27     San Diego
8    Ivan   50        Dallas
9    Judy   29      San Jose
```

## 15 . Matplotlib

Aim : Write a program to draw geometric shapes using matplotlib

```
import matplotlib.pyplot as plt
import numpy as np

# Create a figure and axis
fig, ax = plt.subplots()

# Draw a rectangle
rectangle = plt.Rectangle((0.2, 0.5), 0.4, 0.3, color='blue', alpha=0.3)
ax.add_patch(rectangle)

# Draw a circle
circle = plt.Circle((0.8, 0.2), 0.1, color='green', alpha=0.5)
ax.add_patch(circle)

# Draw a triangle
triangle_points = np.array([[0.6, 0.8], [0.8, 0.8], [0.7, 0.9], [0.6, 0.8]])
triangle = plt.Polygon(triangle_points, closed=True, color='red', alpha=0.4)
ax.add_patch(triangle)

# Set axis limits and labels
ax.set_xlim(0, 1)
ax.set_ylim(0, 1)
ax.set_aspect('equal', adjustable='box')  # Ensure aspect ratio is maintained

# Add grid and labels
ax.grid(True)
ax.set_xlabel('X-axis')
ax.set_ylabel('Y-axis')
```

```
ax.set_title('Geometric Shapes')

# Display the plot
plt.show()
```